สถาปัตยกรรมระบบ (C4): RAG-Diary

> แบบจำลองสถาปัตยกรรมสามระดับ ได้แก่ Context Container และ Component พร้อมตารางเทคโนโลยี ที่ใช้จริง และข้อสังเกตเชิงสถาปัตยกรรมที่ควรปิดในรอบถัดไป

1. Document Control

หัวข้อ	รายละเอียด
ชื่อเอกสาร	สถาปัตยกรรมระบบ (C4)
เวอร์ชัน	1.0
วันที่จัดทำ	15 ตุลาคม 2569
ผู้จัดทำ	Business Analyst / System Analyst (เจ้าของผลงาน)
สถานะ	ผ่านการตรวจทานรอบ 1 (25 ตุลาคม 2569)
เอกสารที่เกี่ยวข้อง	DATA-MODEL, SEQUENCE-DIAGRAMS, ADR, NFR-SPEC

2. Purpose และ Scope

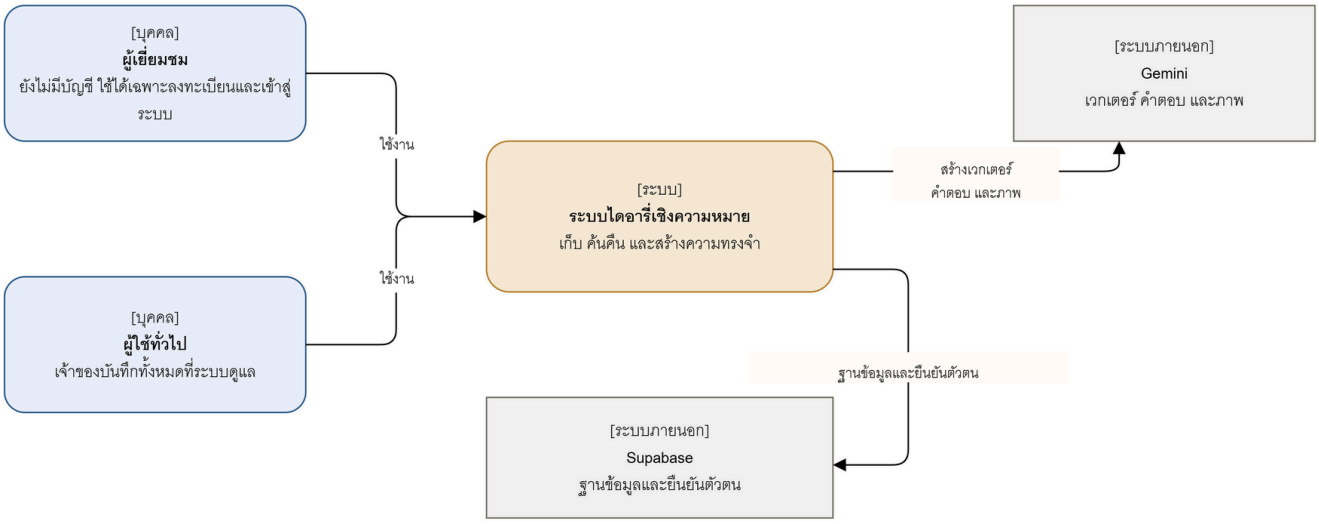
เอกสารนี้ตอบคำถามว่า "ระบบนี้ประกอบด้วยส่วนไหน ส่วนไหนคุยกับส่วนไหน และขอบเขตของแต่ละส่วนคืออะไร" เพื่อให้ผู้อ่านเห็นภาพรวมได้ในหน้าเดียวก่อนลงรายละเอียด

ขอบเขต: สถาปัตยกรรมระดับ Context Container และ Component นอกขอบเขต: model ข้อมูล (อยู่ใน DATA-MODEL) ลำดับข้อความละเอียด (อยู่ใน SEQUENCE-DIAGRAMS) และการตัดสินใจเชิงสถาปัตยกรรมรายข้อ (อยู่ใน ADR)

3. เทคโนโลยีที่ใช้จริง

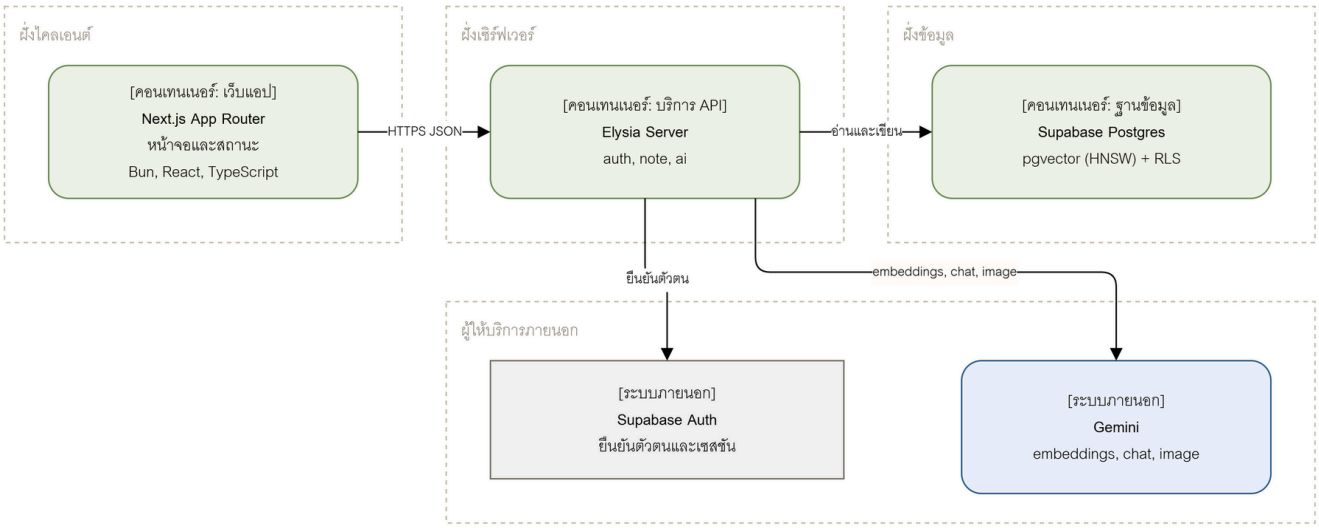
ชั้น	เทคโนโลยี	บทบาท
Frontend	Next.js App Router + Bun + React + TypeScript	เรนเดอร์หน้าจัดการสถานะฝั่งไคลเอนต์
UI	Tailwind CSS + TanStack Query	ออกแบบสไตล์และจัดการข้อมูลแบบ cache
Backend	Elysia + Bun + TypeScript	ให้บริการ API และตรรกะธุรกิจ
Database	Supabase Postgres + pgvector (HNSW)	เก็บข้อมูลและดัชนีเวกเตอร์
Security	Supabase Row Level Security	แยกข้อมูลระดับผู้ใช้
Identity	Supabase Auth	ยืนยันตัวตนและจัดการเซสชัน
AI	Google Gemini (embeddings, chat, image)	สร้างเวกเตอร์ สร้างคำตอบ และสร้างภาพ

4. Level 1: Context



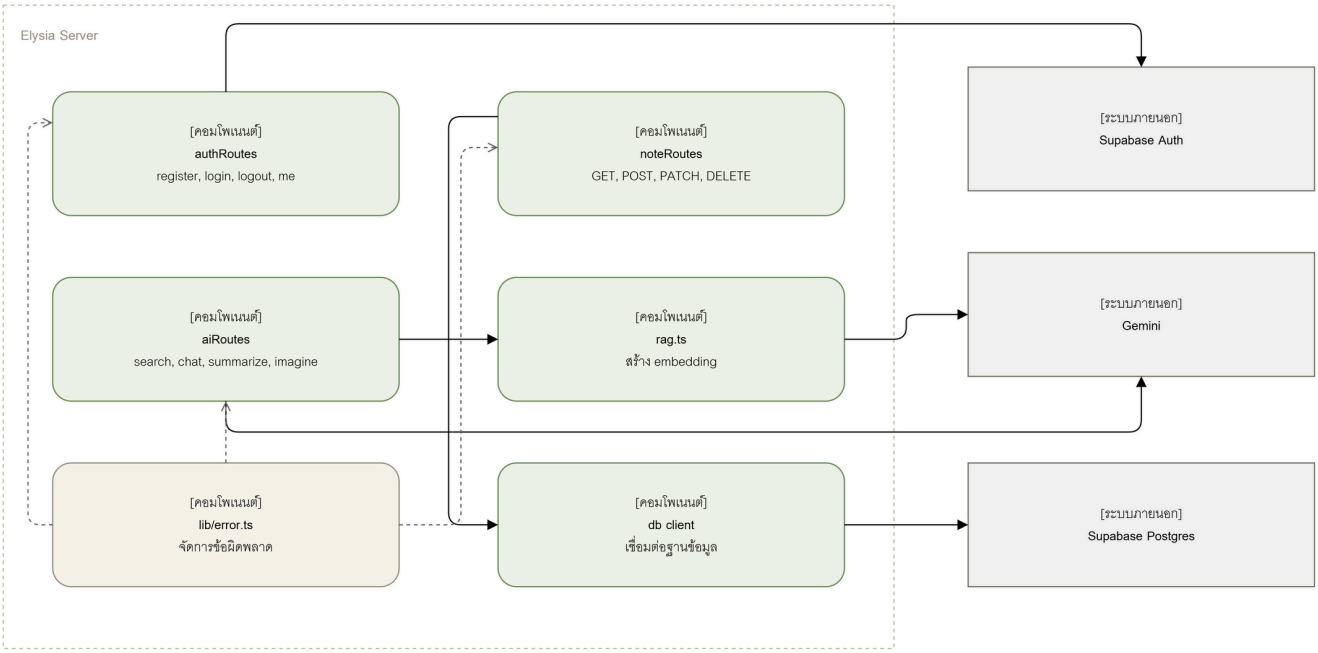
- ผู้เยี่ยมชม: ยังไม่มีบัญชี ใช้ได้เฉพาะการลงทะเบียนและเข้าสู่ระบบ
- ผู้ใช้ทั่วไป: เจ้าของบันทึกทั้งหมดที่ระบบจัดการ
- Google Gemini: ให้บริการสร้างเวกเตอร์ สร้างคำตอบ และสร้างภาพ
- Supabase: ให้บริการฐานข้อมูลและระบบยืนยันตัวตน

5. Level 2: Container



- Frontend: แสดงหน้าจอ จัดการเซสชัน และส่งคำขอ API
- API Server: ตรรกะธุรกิจทั้งหมดของระบบ
- Database: เก็บบันทึก ดัชนีเวกเตอร์ และบังคับการแยกข้อมูลด้วย RLS
- Supabase Auth: ยืนยันตัวตนและออกเซสชัน
- Google Gemini: ประมวลผล AI ทั้งสามด้าน

6. Level 3: Component



Component	ไฟล์	ความรับผิดชอบ	กรณีใช้งานที่เกี่ยวข้อง
authRoutes	modules/user/auth.ts	ลงทะเบียน เข้าสู่ระบบ ออกจากระบบ ตรวจเซสชัน	UC-01
noteRoutes	modules/notes/note.ts	สร้าง อ่าน แก้ไข และลบบันทึก	UC-02, UC-03
aiRoutes	modules/ai/ai.ts	ค้นหาเชิงความหมาย ถามตอบ สรุปอารมณ์ และสร้างภาพ	UC-04, UC-05
rag.ts	modules/ai/rag.ts	สร้าง embedding จากเนื้อหาบันทึก	UC-02
authGuard	lib/auth.ts	ตรวจเซสชันจาก cookie httpOnly (รับ Bearer token เป็นทางถอย) และผูกไคลเอนต์ฐานข้อมูลตามสิทธิ์ผู้ใช้	ทุก UC ที่เข้าถึงข้อมูล
error.ts	lib/error.ts	แปลงข้อผิดพลาดเป็นรูปแบบที่ส่งกลับ API	ทุก UC
db client	lib/db.ts	เชื่อมต่อฐานข้อมูลและรันคำสั่ง	ทุก UC ที่เข้าถึงข้อมูล

7. ข้อสังเกตเชิงสถาปัตยกรรม

ประเด็น	สถานะปัจจุบัน	ข้อเสนอ
รูปแบบระบบ	Modular monolith แยกตาม module ตามหน้าที่	รักษาไว้จนกว่าจะมีเหตุผลให้แยกเป็นบริการ
การเก็บเวกเตอร์	pgvector พร้อมดัชนี HNSW	เหมาะกับสเกลของเวอร์ชัน 1.0
การแยกข้อมูล	บังคับด้วย RLS ที่ฐานข้อมูล	ควรทดสอบการแยกข้อมูลเป็นกรณีพิเศษเสมอ
การเก็บ token	cookie แบบ httpOnly ผ่าน same-origin proxy (ADR-10)	JavaScript อ่านไม่ได้ ต้อง proxy ผ่าน Next.js ทุกคำขอ
การป้องกัน endpoint	จำกัดอัตราต่อผู้ใช้ในโค้ดแล้ว (รอเปิดใช้หลังรัน migration)	ต้องรัน migration ก่อนจึงถือว่าทำงานจริง

8. References

เอกสาร	ความสัมพันธ์
DATA-MODEL	โครงสร้างข้อมูลที่ container จัดการ
SEQUENCE-DIAGRAMS	ลำดับข้อความระหว่าง component
ADR	เหตุผลของการเลือกแต่ละเทคโนโลยี
NFR-SPEC	ตัวชี้วัดที่สถาปัตยกรรมต้องรองรับ